# ITDesk — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

## The final layout

```
<repo folder>/
├── data/kb/
│   ├── printers.md
│   ├── software.md
│   └── vpn.md
├── src/
│   ├── models/__init__.py, llm.py
│   ├── rag/__init__.py, kb_store.py, retriever.py
│   ├── utils/__init__.py, logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── confidence.py
│   ├── exceptions.py
│   ├── graph.py
│   └── state.py
├── tests/
│   ├── integration/test_live_support.py
│   └── unit/test_config.py, test_confidence.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

## Setup files

`requirements.txt`:

```
chromadb
langchain-community
```

```
langchain-openai
langgraph
langgraph-checkpoint-sqlite
openai
python-dotenv
```

`.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`, `*.sqlite`. `LICENSE` is the MIT text. Every folder under `src/` gets an empty `__init__.py`.

## Sample IT KB (`data/kb/`)

```
# data/kb/vpn.md
To connect to the VPN, open the VPN client, sign in with your SSO account,
and choose the office gateway. Contact IT if the client is not installed.
```

```
# data/kb/software.md
To install approved software, open the Company Portal, find the app, and
click Install. Approvals happen automatically for listed apps.
```

```
# data/kb/printers.md
To add a printer, open Settings, choose Printers, and pick the one named for
your floor. The print queue is on the intranet under IT Services.
```

(Leave account-lockout, phishing, and admin-access requests *uncovered*, so they escalate.)

## Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
```

```
class Config:
    openai_api_key: str
    log_level: str
    confidence_cutoff: float

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        confidence_cutoff=float(os.getenv("CONFIDENCE_CUTOFF", "0.8")),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

## The RAG layer and the confidence gate

```
# src/rag/kb_store.py
# Load the IT KB articles, embed them, and store them in Chroma so a question
# can be matched to the closest article by meaning (Doc08).
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings

import chromadb

COLLECTION_NAME = "it_kb"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")

def build_kb_store(kb_dir="data/kb"):
    loader = DirectoryLoader(kb_dir, glob="**/*.md", loader_cls=TextLoader)
    documents = loader.load()
    texts = []
    for document in documents:
        texts.append(document.page_content)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    for index in range(len(texts)):
        ids.append(f"doc-{index}")
    collection.add(ids=ids, documents=texts,
                   embeddings=EMBEDDINGS.embed_documents(texts))
    return collection
```

```
# src/rag/retriever.py
from src.rag.kb_store import EMBEDDINGS

NO_MATCH_DISTANCE = 2.0

def best_match(collection, question):
    if collection.count() == 0:
        return "", NO_MATCH_DISTANCE
    results = collection.query(
        query_embeddings=[EMBEDDINGS.embed_query(question)], n_results=1,
    )
```

```
    documents = results["documents"][0]
    distances = results["distances"][0]
    if not documents:
        return "", NO_MATCH_DISTANCE
    return documents[0], distances[0]
```

```
# src/confidence.py
# The whole bot turns on this: is the best KB match close enough to answer from?

def is_confident(distance, cutoff=0.8):
    return distance <= cutoff
```

## The state and the graph

```
# src/state.py
from typing import TypedDict

class SupportState(TypedDict):
    question: str
    context: str
    best_distance: float
    answer: str
    escalated: bool

def start_state(question):
    return {
        "question": question, "context": "", "best_distance": 2.0,
        "answer": "", "escalated": False,
    }
```

```
# src/graph.py
# retrieve -> route on the confidence gate -> answer (grounded) OR escalate
# (interrupt, pausing for a human).
from langchain_core.prompts import ChatPromptTemplate
from langgraph.graph import END, START, StateGraph
from langgraph.types import interrupt

from src.confidence import is_confident
from src.rag.retriever import best_match
from src.state import SupportState

ANSWER_PROMPT = (
    "Answer the user's IT question using ONLY the knowledge-base article "
```

```
    "below. Be brief and add nothing not in it.\n\nArticle:\n{context}\n\n"
    "Question: {question}"
)

def build_graph(model, collection, cutoff=0.8):
    def retrieve_node(state):
        text, distance = best_match(collection, state["question"])
        return {"context": text, "best_distance": distance}

    def route(state):
        if is_confident(state["best_distance"], cutoff):
            return "answer"
        return "escalate"

    def answer_node(state):
        prompt = ChatPromptTemplate.from_template(ANSWER_PROMPT)
        chain = prompt | model
        reply = chain.invoke({
            "context": state["context"], "question": state["question"],
        })
        return {"answer": reply.content, "escalated": False}

    def escalate_node(state):
        human = interrupt({"question": state["question"]})
        return {"answer": human, "escalated": True}

    graph = StateGraph(SupportState)
    graph.add_node("retrieve", retrieve_node)
    graph.add_node("answer", answer_node)
    graph.add_node("escalate", escalate_node)
    graph.add_edge(START, "retrieve")
    graph.add_conditional_edges("retrieve", route,
                                {"answer": "answer", "escalate": "escalate"})
    graph.add_edge("answer", END)
    graph.add_edge("escalate", END)
    return graph
```

## main.py

```
# main.py
import sys

from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command
```

```
from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.kb_store import build_kb_store
from src.state import start_state

def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your question"')
        return
    question = sys.argv[1]

    config = load_config()
    model = get_model(config)
    collection = build_kb_store()
    app = build_graph(model, collection, config.confidence_cutoff).compile(
        checkpointer=MemorySaver())

    thread = {"configurable": {"thread_id": "cli-session"}}
    result = app.invoke(start_state(question), thread)

    if app.get_state(thread).next:                # escalated and paused
        print("ITDesk: I'm not confident I can answer that from the IT "
              "knowledge base.\n  -> Escalating to an IT staffer...")
        human = input("  [IT staffer, type a reply]: ")
        result = app.invoke(Command(resume=human), thread)
        print(f"ITDesk (via human): {result['answer']}\n[escalated]")
    else:
        print(f"ITDesk: {result['answer']}")

if __name__ == "__main__":
    main()
```

## The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "ITDESK_TEST_SETTING"

def test_missing_setting():
```

```
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_confidence.py
# Run from the repo root: python -m tests.unit.test_confidence
from src.confidence import is_confident

def test_small_distance_is_confident():
    result = "OK"
    if not is_confident(0.2, cutoff=0.8):
        result = "WRONG"
    print(f"case 1 (small -> confident)   -> {result}")

def test_large_distance_is_not():
    result = "OK"
    if is_confident(1.5, cutoff=0.8):
        result = "WRONG"
    print(f"case 2 (large -> escalate)    -> {result}")

def test_cutoff_itself_is_confident():
    result = "OK"
    if not is_confident(0.8, cutoff=0.8):
        result = "WRONG"
    print(f"case 3 (== cutoff -> confident) -> {result}")

if __name__ == "__main__":
    test_small_distance_is_confident()
    test_large_distance_is_not()
    test_cutoff_itself_is_confident()
```

```
# tests/integration/test_live_support.py
# Real model + RAG + graph. Run from the repo root:
#   python -m tests.integration.test_live_support
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command

from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.kb_store import build_kb_store
from src.state import start_state

def build_app():
    config = load_config()
    model = get_model(config)
    collection = build_kb_store()
    graph = build_graph(model, collection, config.confidence_cutoff)
    return graph.compile(checkpointer=MemorySaver())

def test_covered_is_answered():
    app = build_app()
    thread = {"configurable": {"thread_id": "t-answer"}}
    result = app.invoke(start_state("How do I connect to the VPN?"), thread)
    ok = not app.get_state(thread).next and "vpn" in result["answer"].lower()
    print(f"case 1 (covered answered) -> {'OK' if ok else 'WRONG'}")

def test_sensitive_escalates():
    app = build_app()
    thread = {"configurable": {"thread_id": "t-escalate"}}
    app.invoke(start_state("My account is locked, reset it"), thread)
    paused = bool(app.get_state(thread).next)
    result = app.invoke(Command(resume="I'll verify and unlock it."), thread)
    ok = paused and result["escalated"] is True
    status = "OK" if ok else "WRONG"
    print(f"case 2 (sensitive escalates + resumes) -> {status}")

if __name__ == "__main__":
    test_covered_is_answered()
    test_sensitive_escalates()
```

**Expected unit output:**

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (small -> confident)   -> OK
case 2 (large -> escalate)    -> OK
case 3 (== cutoff -> confident) -> OK
```

## The whole project, in one sentence

A RAG search over the IT knowledge base, a deterministic confidence gate, and a real human handoff ( `interrupt` + checkpointer) — so routine questions get answered from the KB and sensitive ones genuinely pause for a human. The same patterns as DeflectDesk, on an IT helpdesk where answering the wrong thing is a security risk.